

CASE STUDY

QuickMeds

Online Medicine Delivery System

Feature Focus: Shopping Cart, Order Management & Smart Budget Optimizer

Student Name: **Tanmay Warthe**

Class: **CT-B** | Roll Number: **178**

Project Type: **Front-End Web Application**

Technology Stack: **HTML5, CSS3, JavaScript, LocalStorage API**

NEW

0/1 Knapsack Dynamic Programming Algorithm

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Proposed Solution
5. Features
6. System Modules
7. Feature Deep Dive – Shopping Cart & Order Management
8. 0/1 Knapsack Budget Optimizer – Algorithm & Implementation
9. Technology Stack
10. Working of the System
11. Advantages
12. Limitations
13. Future Scope
14. Project Output Screenshots
15. Conclusion

1. Introduction

QuickMeds is a web-based Online Medicine Delivery System developed as a research paper implementation assignment. The platform enables users to browse a catalogue of medicines, search for specific products, manage a shopping cart, and place orders — all from within a browser, with no server-side dependency.

The project draws its conceptual foundation from a research paper published in IRJMETS (International Research Journal of Modernization in Engineering Technology and Science), titled *Online Medicine Delivery System / E-Pharmacy Management System*. The front-end implementation translates the proposed system architecture into a functional, user-accessible prototype.

This case study specifically analyses the Shopping Cart, Order Management, and the Smart Budget Optimizer feature of QuickMeds — the last of which employs the classic **0/1 Knapsack Dynamic Programming Algorithm** to recommend an optimal set of medicines within a user-defined budget.

2. Problem Statement

Access to essential medicines remains a challenge in many urban and semi-urban areas of India due to store timings, physical distance, and the difficulty of tracking medicine availability. Traditional pharmacy visits require the customer to be present in person, which is often inconvenient for elderly patients, working professionals, or people with mobility limitations.

Existing e-commerce platforms either do not specialise in medicines or present a complex user interface that is not beginner-friendly. There is a clear gap for a dedicated, lightweight, and easy-to-use online pharmacy interface that allows users to:

- Browse and search medicines without requiring an account.
- Add multiple items to a cart and modify quantities in real time.
- Review a complete order summary with taxes and shipping before placing an order.
- Experience a smooth, responsive interface on both desktop and mobile devices.
- Receive intelligent budget-based medicine recommendations using an optimisation algorithm.

3. Objectives

1. Develop a clean, responsive multi-page pharmacy website using HTML5, CSS3, and JavaScript.
2. Implement a fully functional product catalogue with search and filter capabilities.
3. Build a persistent shopping cart using the browser's LocalStorage API.
4. Calculate and display a dynamic order summary including subtotal, 5% GST, and shipping.
5. Implement the **0/1 Knapsack Dynamic Programming Algorithm** as a Smart Budget Optimizer.
6. Provide a user authentication interface with client-side form validation.
7. Ensure mobile responsiveness and cross-browser compatibility.

4. Proposed Solution

QuickMeds proposes a front-end-only web application structured across four linked HTML pages. The solution deliberately avoids any back-end dependency, making it deployable on any static file host or directly from a local file system — ideal for a prototype or academic demonstration.

The core of the proposed solution is the LocalStorage-backed cart engine. When a user clicks "Add to Cart" on any product card, the item is serialised as a JSON object and written to the browser's LocalStorage. The cart page reads this data on load, renders each item dynamically, and recalculates totals whenever the user modifies a quantity or removes a product.

An additional intelligent layer — the **Smart Budget Optimizer** — uses the **0/1 Knapsack Dynamic Programming Algorithm** to recommend the combination of medicines that maximises total health-priority score without exceeding the user's entered budget.

5. Features

Feature	Description
Product Catalogue	Eight medicine products in a responsive grid with name, description, and price.
Search Functionality	Home-page search bar filters products using LocalStorage as a cross-page relay.
Add to Cart	Re-adding an existing item increments its quantity instead of creating a duplicate entry.
Cart Management	Increase/decrease quantity with +/- controls or remove items entirely; UI updates in real time.
Order Summary	Dynamic display of subtotal, ₹40 shipping, 5% GST, and grand total.
Checkout Flow	Confirms order, clears cart, and redirects to home page.
Smart Budget Optimizer	0/1 Knapsack algorithm recommends the optimal medicine set within user's budget.
User Authentication UI	Dual-panel login page with client-side password-match validation.
Responsive Design	CSS Flexbox and Grid layout adapts to mobile, tablet, and desktop screens.

6. System Modules

6.1 Home Module (index.html)

Landing page with QuickMeds branding, search bar, and featured products. Handles search initiation by storing the query in LocalStorage and redirecting to the products page.

6.2 Product Catalogue Module (products.html)

Displays the complete inventory of eight medicines in a card grid. Reads a pending search term from LocalStorage on page load and filters cards accordingly.

6.3 Cart & Order Module (cart.html)

The most feature-rich module. Renders cart items with quantity controls, computes the live order summary, hosts the Smart Budget Optimizer (Knapsack), and handles the checkout action.

6.4 Authentication Module (login.html)

Side-by-side Sign In and Sign Up forms. Sign Up validates that passwords match before confirming account creation. Both forms use HTML5 validation attributes.

7. Feature Deep Dive – Shopping Cart & Order Management

7.1 Data Storage Architecture

Cart data is stored as a JSON array in the browser's LocalStorage under the key *cart*. Each element is an object with three properties:

Property	Type	Description
name	String	Product name — used as the unique identifier
price	Number	Unit price in Indian Rupees (₹)
quantity	Number	Number of units the user intends to purchase

7.2 Add to Cart Logic

When the user clicks "Add to Cart", the *addToCart()* function: (1) reads the existing cart array from LocalStorage, (2) searches for an entry with a matching name, (3) increments quantity if found, or pushes a new object with quantity 1 if not found, and (4) writes the updated array back to LocalStorage.

7.3 Cart Rendering

The *loadCart()* function on *cart.html* reads the cart array on page load. If empty, an "empty cart" message is rendered. Otherwise, each item is rendered as a *cart-item* div containing product name, unit price, quantity controls, line total, and a Remove button.

7.4 Quantity Management

The `updateQuantity(index, change)` function adds +1 or -1 to the item at the given index. If the resulting quantity drops to zero, the item is removed via `splice()`. The cart is saved back to `LocalStorage` and `loadCart()` re-renders the UI.

7.5 Order Summary Calculation

Field	Calculation
Subtotal	Sum of (price × quantity) for all items
Shipping	■40 flat (■0 if cart is empty)
GST (5%)	5% of the Subtotal
Grand Total	Subtotal + Shipping + GST

7.6 Checkout

The `proceedToCheckout()` function verifies the cart is non-empty, shows a success alert, clears the `cart` key from `LocalStorage`, and redirects the user to the home page — simulating a completed order placement.

8. 0/1 Knapsack Budget Optimizer – Algorithm & Implementation

■ Smart Budget Optimizer

0/1 Knapsack · Dynamic Programming · $O(n \times W)$

8.1 Background & Motivation

A patient visiting an online pharmacy often has a fixed budget but wants to maximise the health benefit of their purchase. Selecting medicines manually is error-prone and sub-optimal. The classic **0/1 Knapsack Problem** — a fundamental combinatorial optimisation problem — maps perfectly to this scenario:

- **Items** → medicines in the catalogue
- **Weight** → price of each medicine (■)
- **Value** → health-priority score (1–10) assigned to each medicine
- **Capacity** → the user's stated budget (■)
- **Constraint** → each medicine can be selected at most once (0/1 constraint)

8.2 Medicine Catalogue with Priority Scores

The following eight medicines form the knapsack item set. Health-priority scores are pre-assigned based on the commonality and criticality of each medicine:

#	Medicine	Price (■)	Priority Score	Rationale
1	Paracetamol 500mg	45	9	Essential pain & fever relief
2	Vitamin C Tablets	120	7	Immunity booster
3	Cough Syrup	85	8	Common ailment treatment
4	Antiseptic Cream	65	8	First-aid essential
5	Multivitamin Capsules	250	6	General supplement
6	Pain Relief Gel	95	7	Muscle & joint pain
7	Calcium Tablets	180	6	Bone health
8	Digestive Tablets	55	7	Common digestive need

8.3 Problem Definition

Given a set of n medicines, each with a price p_i and a health-priority value v_i , and a budget W , find the subset $S \subseteq \{1 \dots n\}$ such that:

Objective:	Maximise $\sum v_i$ (for i in S)
Subject to:	$\sum p_i \leq W$ (for i in S)
Constraint:	Each item selected at most once (0/1)

8.4 Algorithm: Bottom-Up Dynamic Programming

The 0/1 Knapsack problem is solved using a bottom-up dynamic programming approach. A 2D DP table is constructed where each cell stores the maximum achievable health-priority value for a given number of items and budget.

Step 1 — Initialisation

Create a 2D table $dp[0\dots n][0\dots W]$ initialised to 0.

$dp[i][w]$ = maximum health-priority value achievable using the first i medicines within a budget of w rupees.

$dp[0][w] = 0$ for all w (no items selected)

$dp[i][0] = 0$ for all i (zero budget selects nothing)

Step 2 — Recurrence Relation

For each medicine i (1 to n) and each budget w (0 to W):

Case A — Item too expensive ($price[i] > w$):

```
dp[i][w] = dp[i-1][w] ← skip item i
```

Case B — Item fits ($price[i] \leq w$):

```
dp[i][w] = max(  
    dp[i-1][w], ← skip item i  
    dp[i-1][w - price[i]] + value[i] ← include item i  
)
```

Step 3 — Optimal Value

After filling the entire table, $dp[n][W]$ holds the maximum total health-priority score achievable within the full budget W using all n medicines.

Step 4 — Backtracking to Recover Selected Items

Starting from cell $dp[n][W]$ and moving upward, an item i is identified as selected if $dp[i][w] \neq dp[i-1][w]$. The remaining budget is then reduced by $price[i]$ and the process continues until $i = 0$.

```
w = W  
for i = n down to 1:  
    if dp[i][w] ≠ dp[i-1][w]:  
        selected.add(item[i])  
        w = w - price[i]
```

8.5 Complexity Analysis

Metric	Value	Details
Time Complexity	$O(n \times W)$	$n = 8$ medicines, $W =$ user's budget in ₹
Space Complexity	$O(n \times W)$	2D DP table of size $(n+1) \times (W+1)$
Optimality	Guaranteed	DP explores all subsets implicitly
n (items)	8	Fixed catalogue of eight medicines

8.6 Worked Example

Suppose the user enters a budget of ₹200. The algorithm evaluates all $2^8 = 256$ possible subsets implicitly via the DP table and identifies the optimal selection:

Medicine	Price (₹)	Priority	Selected?
Paracetamol 500mg	45	9	Yes
Cough Syrup	85	8	Yes
Antiseptic Cream	65	8	Yes
Digestive Tablets	55	7	✗ No (budget exhausted)

Result: 3 medicines selected | Total cost ₹195 / ₹200 | Total health-priority score = 25 | Budget remaining ₹5

8.7 UI Integration in cart.html

The Smart Budget Optimizer is embedded directly in the cart page. The user flow is as follows:

1. User enters a budget amount (₹) in the input field.
2. Clicks "Find Best Combo" — triggers `runKnapsack()`.
3. The function validates the input, then calls `knapsack01(CATALOGUE, budget)`.
4. The returned optimal array is rendered as coloured medicine "pills" showing name, price, and priority.
5. Summary stats display: items selected, total cost vs budget, total priority score, remaining budget.
6. The "+ Add All to Cart" button adds every optimal medicine to the cart in one click.

8.8 Why 0/1 Knapsack is the Right Algorithm Here

Approach	Handles 0/1?	Optimal?	Complexity	Suitability
Greedy (by priority/price)	No	Not guaranteed	$O(n \log n)$	Simple but sub-optimal
Brute Force	Yes	Yes	$O(2^n)$	Infeasible for large n
0/1 Knapsack DP	Yes	Yes	$O(n \times W)$	Optimal & efficient
Fractional Knapsack	No	N/A	$O(n \log n)$	Cannot split medicines

9. Technology Stack

Technology	Purpose
HTML5	Page structure and semantic markup across all four pages
CSS3	Styling, Flexbox/Grid layout, responsive breakpoints, animations
JavaScript (ES6+)	Cart logic, DOM manipulation, LocalStorage API, Knapsack algorithm
LocalStorage API	Client-side persistent storage for cart items and search queries
Google Fonts (Poppins)	Typography for a modern, readable UI
CSS Custom Properties	Consistent colour theming across the entire application

10. Working of the System

1. User opens index.html and is greeted by the QuickMeds banner and featured products.
2. User types a medicine name into the search bar and clicks "Search". The term is saved to LocalStorage and the user is redirected to products.html.
3. On products.html, matching products are displayed. The user clicks "Add to Cart" on one or more products.
4. User navigates to cart.html. All previously added items appear with quantities and individual totals.
5. User adjusts quantities using the +/- buttons. The order summary updates instantly.
6. **[Optional]** User enters a budget in the Smart Budget Optimizer and clicks "■ Find Best Combo". The 0/1 Knapsack algorithm computes and displays the optimal medicine selection.
7. User clicks "Proceed to Checkout". An order confirmation message is shown, the cart is cleared, and the user is returned to the home page.

11. Advantages

- No Installation Required – Runs entirely in the browser; no server, database, or runtime environment needed.
- Persistent Cart – LocalStorage retains cart contents even across page refreshes.
- Real-Time Updates – Order summary recalculates immediately on every user interaction.
- Lightweight & Fast – No external API calls or heavy frameworks.
- Algorithmically Intelligent – The 0/1 Knapsack optimizer delivers provably optimal medicine recommendations within any given budget.
- Responsive Design – Flexbox and Grid layout adapts to all screen sizes.
- Beginner-Friendly UI – Clean visual hierarchy and colour-coded states.

12. Limitations

- Front-End Only – No real order processing, payment gateway, or inventory management.
- No User Authentication Backend – The login form validates only on the client side.
- LocalStorage Constraints – Cart data is browser- and device-specific; not shareable across sessions.
- No Prescription Upload – Real pharmacy systems require prescription verification for certain medicines.
- Limited Product Catalogue – Only eight hardcoded products; no admin panel to add inventory.
- No Stock Management – The system does not track or enforce product availability limits.
- Static Priority Scores – Health-priority values are hardcoded; a real system would derive them dynamically.

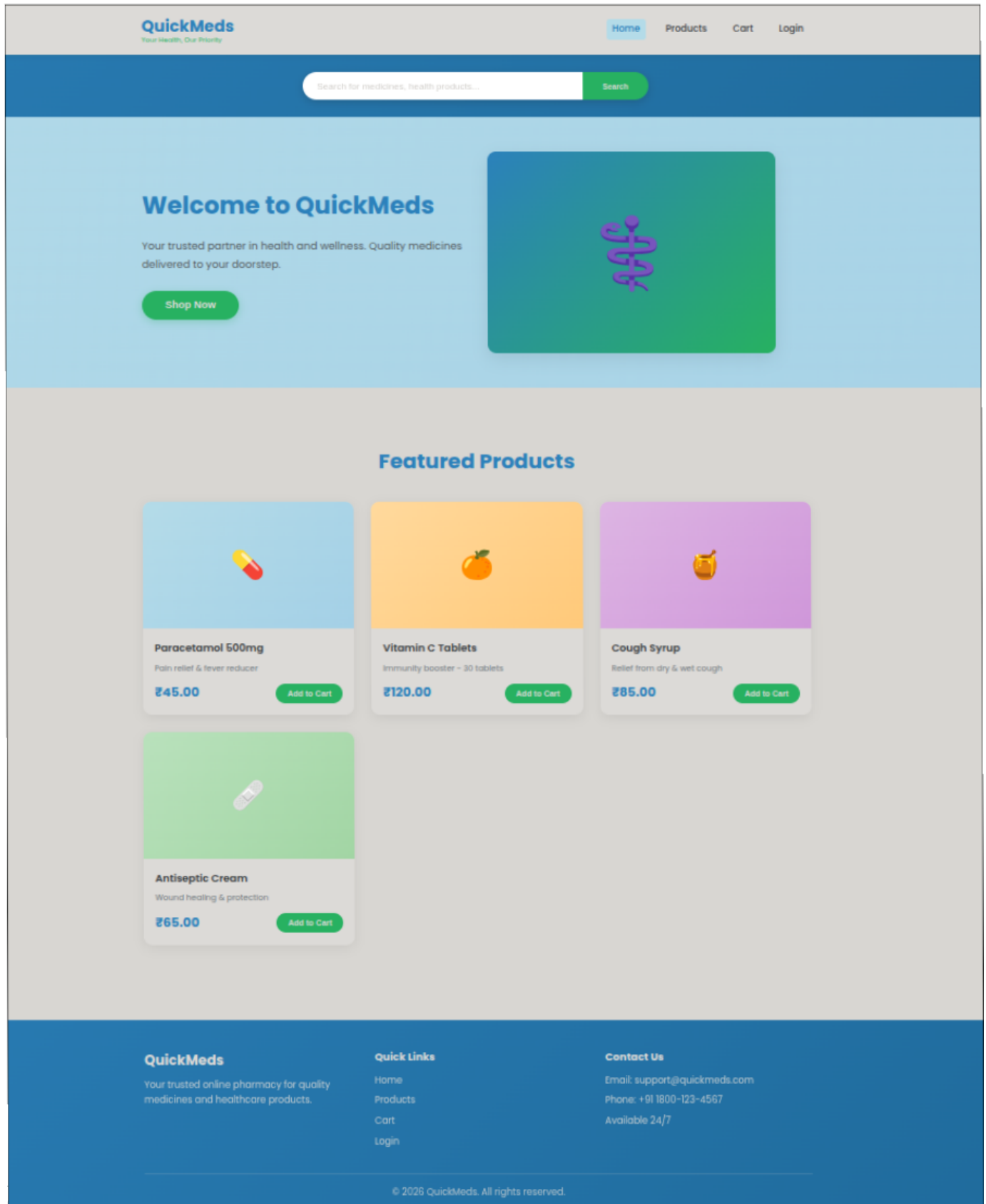
13. Code & Algorithm

```
JS knapsack.js > knapsack01
1  function knapsack01(items, budget) {
2      const n = items.length;
3      const W = budget;
4
5      const dp = Array.from({ length: n + 1 }, () => new Array(W + 1).fill(0));
6
7      for (let i = 1; i <= n; i++) {
8          const item = items[i - 1];
9          for (let w = 0; w <= W; w++) {
10             dp[i][w] = dp[i - 1][w];
11
12             if (item.price <= w) {
13                 const valueIfIncluded = dp[i - 1][w - item.price] + item.priority;
14                 if (valueIfIncluded > dp[i][w]) {
15                     dp[i][w] = valueIfIncluded;
16                 }
17             }
18         }
19     }
20
21     const selected = [];
22     let w = W;
23     for (let i = n; i >= 1; i--) {
24         if (dp[i][w] !== dp[i - 1][w]) {
25             selected.push(items[i - 1]);
26             w -= items[i - 1].price;
27         }
28     }
29
30     return selected;
31 }
32
33 if (typeof module !== 'undefined' && module.exports) {
34     module.exports = { knapsack01 };
35 }
36
```

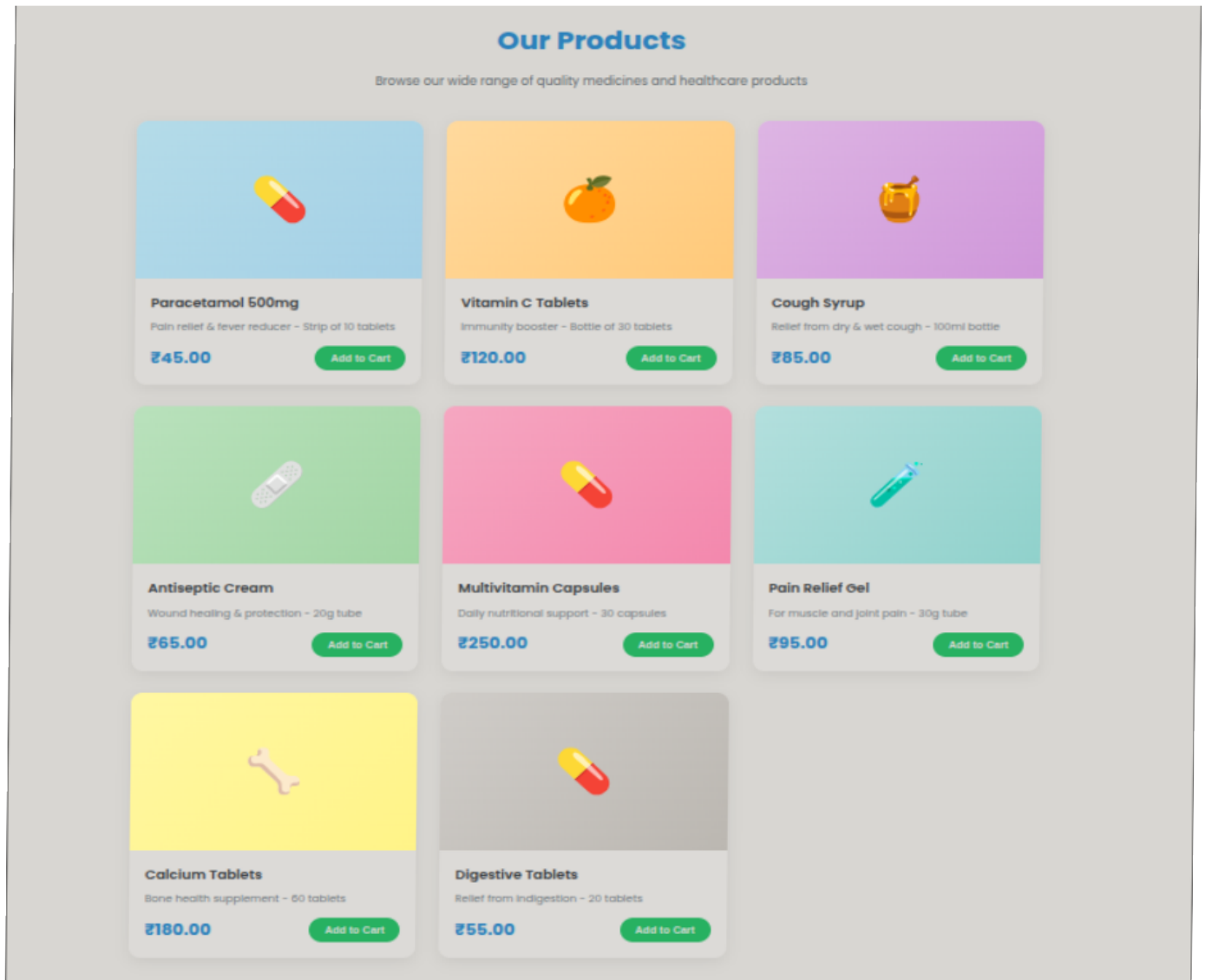
14. Project Output Screenshots

The following section contains screenshots of the QuickMeds application capturing each major feature and page. Insert your project output images in the spaces provided below.

Home Page



Product Page



Login /Signup Page

Sign In

Email Address

Password

☐ Remember me

Sign In

Don't have an account? Sign up on the right →

Sign Up

Full Name

Email Address

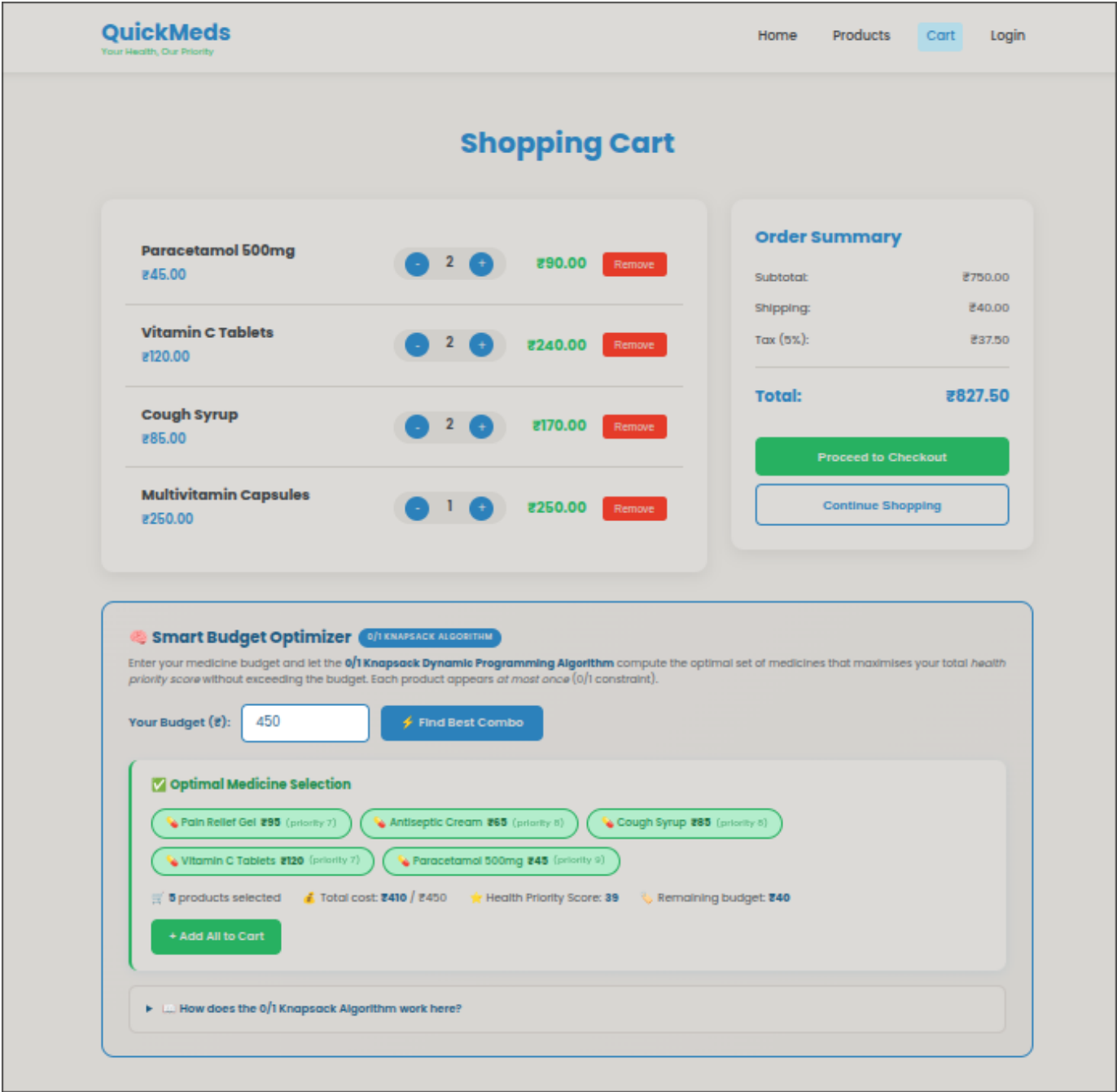
Phone Number

Password

Confirm Password

Sign Up

Cart Page



15. Conclusion

QuickMeds successfully demonstrates a complete front-end implementation of an Online Medicine Delivery System. The Shopping Cart and Order Management feature is built on a clean, well-structured LocalStorage architecture that delivers a smooth, real-time user experience without any server dependency.

The standout addition to this implementation is the **Smart Budget Optimizer**, which leverages the **0/1 Knapsack Dynamic Programming Algorithm** to solve a genuine combinatorial optimisation problem in real time inside the browser. By assigning health-priority scores to medicines and treating the user's budget as the knapsack capacity, the feature provides provably optimal medicine recommendations — demonstrating how classical algorithms can deliver meaningful value in everyday applications.

The project fulfils all its stated objectives: product browsing, cross-page search, persistent cart management, dynamic order calculation, intelligent budget optimisation, and a simulated checkout flow are all implemented and functional. The codebase is modular, with each page handling its own concerns while sharing a common design language and utility functions.

While the current implementation is intentionally scoped as an academic prototype, the design decisions made — separation of concerns, local persistence, responsive layout, and algorithmic intelligence — provide a solid foundation for future full-stack development.

Submitted by: Tanmay Warthe | Class: CT-B | Roll No: 178